

WEEK 5 · DAY 1

AI Agent Foundations

How a Single Agent Plans, Remembers, and Invokes Tools — Before It Collaborates With Others

1-Hour Lecture Session — Concepts & Architecture

Rajesh Pasham

Program Context

How today's session fits the wider 8-week delivery

Session Type	Concepts and architecture lecture
Platform	Individual Palantir Developer Tier accounts, in continuous use since Week 1, Day 3
This Week's Lab (Day 3)	Multi-Agent Insurance Workflow — orchestrated agents that collaborate, invoke Actions and Functions, and write back to your Ontology
Skill Gained This Week	AI Agent Engineer — the fifth rung of the program's Executive Skills Matrix

Where Today Fits

From one copilot answering questions, to agents that act on your behalf

Week 2 gave every batch a Claims Copilot that could retrieve knowledge and answer questions. Week 3 gave it hands: Actions and Functions that write back to the Ontology, with a human approving every step. Week 5 is where those two things merge — a system that plans its own sequence of tool calls, decides which Action or Function to invoke, and only pauses for a human where your governance model says it must.

Not new theory in isolation — it is Weeks 2 through 4, recombined.

The Claims Copilot's retrieval (Week 2), the Ontology Actions and Functions it can now call (Week 3), and the document-derived knowledge it can reason over (Week 4) are exactly the building blocks an agent orchestrates. Keep Week 2–4's exercises open as your reference for what each tool actually does.

Today's Roadmap

Four topic groups, building toward Wednesday's multi-agent lab

1

From Copilot to Agent

What actually changes when a system starts planning its own steps instead of answering one prompt at a time

2

Agent Architecture

Planning, memory, tool invocation, guardrails, and error handling — the parts every agent needs

3

Enterprise Agent Design Patterns

Event-driven triggers, microservices-style isolation, and where these ideas live inside AIP's Agent Studio

4

Insurance Agent Roles

The specialized agents your batch's lab will orchestrate, and why one agent per responsibility beats one large agent

By the End of Today

The vocabulary Wednesday's multi-agent lab assumes

- ✓ **Explain what makes something an “agent” rather than a copilot**

Autonomy over a sequence of steps, not just a single grounded answer

- ✓ **Describe the plan → act → observe loop**

And why an agent re-plans after every tool call instead of following a fixed script

- ✓ **Distinguish short-term working memory from persisted long-term memory**

And why an agent forgetting between runs is sometimes the safer default

By the End of Today

The vocabulary Wednesday's multi-agent lab assumes (cont'd)

✓ **Explain how Functions and Actions become an agent's “tools”**

The same building blocks from Week 3 — invoked by the agent instead of a fixed workflow

✓ **Describe event-driven triggers and microservices-style agent isolation**

The enterprise patterns behind how agents are deployed and secured, not just how they reason

✓ **Name the seven insurance agents and your batch's two-agent focus**

So Wednesday's lab starts from a shared vocabulary, not a blank page

TOPIC 1

From Copilot to Agent

What actually changes when a system starts planning its own steps

A Quick Recap: What Week 2's Copilot Actually Did

Grounding the comparison in something you already built

Retrieval, then generation

Semantic search pulled relevant Ontology objects and document chunks; the LLM used them as context to produce one grounded answer.

A single request → single response cycle

An adjuster asked a question, the Copilot answered it — the interaction was complete the moment a response was returned.

No memory of its own decisions

The Copilot didn't track what it had already told the adjuster, or whether that answer led to any downstream action.

What Made It Not Yet an “Agent”

A grounded answer is not the same as an executed plan

It answered one question at a time

The Claims Copilot retrieved relevant knowledge with Semantic search and produced a single grounded response — powerful, but the workflow began and ended in one exchange.

It never decided what to do next on its own

Any writeback that followed (Week 3's Ontology Actions) was triggered by a human choosing to submit it — the Copilot itself proposed an answer, it did not choose or sequence actions.

Nothing tracked whether the goal was actually achieved

There was no loop checking the outcome of a step and adjusting course — each interaction was self-contained, with no notion of an ongoing objective.

Defining an Agent

Autonomy over a sequence of steps toward a goal, not a single response

Goal-directed, not turn-directed

An agent is given an objective (“validate this claim's coverage and flag anomalies”), not a single prompt — it determines how many steps that takes.

It chooses its own tools

Given a goal and a set of available Functions and Actions, the agent decides which to call, in what order, based on what it learns along the way.

It re-plans as it goes

After each tool call, the agent observes the result and decides the next step — closer to how a human adjuster works a claim than to a fixed checklist.

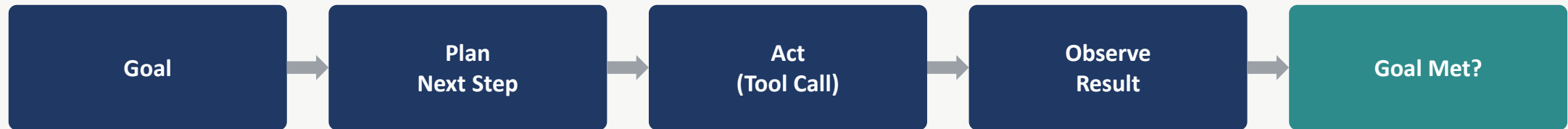
Copilot vs. Workflow vs. Agent

Three patterns you've now built — knowing which one you're looking at matters

Dimension	Copilot (Week 2)	Fixed Workflow (Week 3)	Agent (Week 5)
Trigger	A human asks a question	A defined event, every time	A goal, decomposed at runtime
Step sequence	One retrieval + one answer	Fixed, identical every run	Decided and revised by the agent
Tool choice	None — retrieval only	Predetermined by the workflow author	Chosen by the agent from what's available
Adapts mid-run	No	No	Yes — replans after each observation
Stops when	One answer is returned	The last defined step completes	The goal is met, or a human is needed

The Reasoning Loop

Plan → Act → Observe — repeated until the goal is met or a human is needed



This loop is what separates an agent from a workflow. A fixed AIP Logic workflow from Week 3 executes the same steps every run; an agent decides its steps at runtime, based on what each tool call returns — including deciding to stop and hand off to a human, per your batch's governance model.

Walking the Loop With a Real Claim

A first notice of loss, worked step by step by a Claims Agent

Goal received: “Assess FNOL-2201 and recommend next action”

1

The agent is not told which checks to run — only the outcome expected of it.

Plan: check policy status first

2

The agent reasons that coverage validity gates everything else, so it plans that step first.

Act: call the Coverage Validation Function

3

The same Function type built in Week 3 — invoked here by the agent's own choice, not a fixed sequence.

Walking the Loop With a Real Claim

A first notice of loss, worked step by step by a Claims Agent (cont'd)

Observe: policy is active, but claim amount exceeds the policy's rating tier

4

This observation changes the plan — the next step is no longer a routine approval path.

Plan again: route to the Fraud Detection Agent before proceeding

5

The agent recognizes this pattern warrants collaboration — a preview of Day 2's multi-agent orchestration.

Stop: hand off to a human adjuster for the high-value referral

6

The agent recognizes the boundary set by your batch's governance plan and pauses rather than acting further.

When an Agent Should Stop and Ask a Human

Autonomy is not the same as unlimited authority

A monetary threshold is crossed

The same `isHighValueClaim` / `isHighRiskApplication` boolean pattern from Week 1 can gate not just a security policy, but an agent's authority to act without approval.

The agent's confidence in its own plan is low

An agent that cannot resolve conflicting signals (a valid policy but a suspicious claim pattern) should escalate rather than guess.

The action is irreversible or hard to reverse

Settlement authorization and policy cancellation are exactly the kind of writebacks Week 3's rollback-strategy discussion was preparing you for.

TOPIC 2

Agent Architecture

Planning, memory, tool invocation, guardrails, and error handling

The Three Parts of Every Agent

Whatever it automates, an agent needs all three — none of them optional

Planning

How the agent turns a goal into an ordered (and revisable) sequence of steps.

Memory

What the agent carries forward — within a single run, and, deliberately, across runs.

Tool invocation

How the agent actually does something — by calling a Function, an Action, or a search, and reading back the result.

Planning: Decomposing a Goal Into Steps

Before any tool is called, the agent has to decide what steps the goal requires

A goal is not a step-by-step recipe

“Determine whether this claim is covered” could mean checking the policy, checking exclusions, checking claim history, or all three — the agent decomposes the goal into a candidate sequence.

Planning is revised after every observation

If checking the policy reveals a lapsed payment, the plan changes — the next step is no longer coverage validation, it's a referral, exactly as Week 1's Referral object type anticipated.

Planning is bounded by what tools exist

An agent can only plan around the Functions and Actions actually exposed to it — the same access-control discipline from Week 1's RBAC/ABAC governs which tools an agent may even see.

Two Planning Strategies

Not every agent needs to think several steps ahead before acting

Strategy	How It Decides	Best Suited To
Reactive (single-step) planning	Chooses the single next best action given the current state, without projecting further ahead	Well-understood, low-ambiguity tasks — routing a document by its classification
Deliberate (multi-step) planning	Projects a candidate sequence of several steps before acting, and revises the whole sequence as new information arrives	Ambiguous, multi-signal tasks — assessing whether a claim is fraudulent

Memory: What an Agent Carries Between Steps

Two different kinds of memory, with very different governance implications

Short-term (working) memory

What the agent has learned so far in this run — the claim it's investigating, the coverage result it already retrieved. Scoped to a single run, discarded when the run ends.

Long-term (persisted) memory

State an agent deliberately keeps across runs — for example, a Fraud Detection Agent remembering a pattern seen across multiple prior claims. This is a design decision with real governance cost, not a default.

Default to short-term unless the use case demands otherwise

Persisted memory means deciding what's retained, where it's stored, who can read it, and how long it lives — questions Week 6's LLMOps and governance topics return to in depth.

Memory in Practice: A Fraud Agent Example

The same decision, made twice, illustrates why persisted memory is a choice, not a default

Without long-term memory

Each claim is assessed independently — a single suspicious claim from a policyholder is evaluated on its own facts alone, with no view of that policyholder's claim history.

With long-term memory, deliberately scoped

The Fraud Detection Agent retains a policyholder-level pattern (three claims in six months from the same repair shop) — but only that pattern, retained under an explicit retention policy, not an open-ended transcript of every prior interaction.

The governance question either way

Who approved this retention, how long is it kept, and can the policyholder's data be purged on request — the same questions Week 1's Markings and security policies were built to answer for structured data.

Tool Invocation: Functions and Actions as an Agent's Tools

Nothing new to build here — an agent calls the exact mechanisms from Week 3

A Function is a tool the agent can call

The same AIP Logic Functions from Week 3 — an agent invokes one the way a workflow step did, but the agent decides when and whether to call it.

An Action is how an agent writes back

Ontology Actions remain the only path to a committed change — an agent proposing an Action still goes through the same Object Explorer submission and, where your governance model requires it, the same human approval.

Tool invocation is where autonomy meets governance

Giving an agent access to a tool is a security-policy decision, using the same RBAC/ABAC and Marking patterns from Week 1 — an agent should only ever see the tools its role actually needs.

Tool Selection: How an Agent Chooses the Right Tool

An agent doesn't guess — it matches its goal against what each tool is described to do

Every tool needs a clear description

A Function or Action exposed to an agent carries a description of what it does and what inputs it needs — an agent reasons over these descriptions the same way it reasons over retrieved knowledge.

Ambiguous tool names cause wrong calls

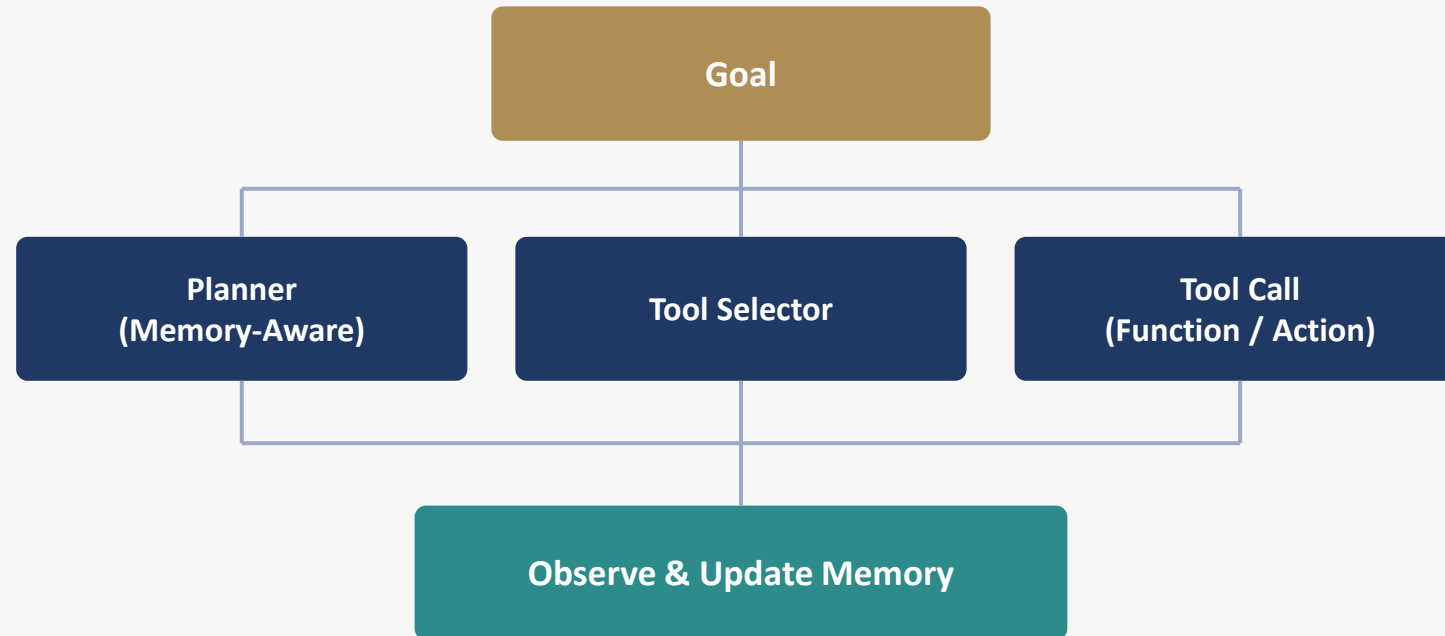
Two Functions both named “UpdateStatus” force the agent to guess — the same naming discipline you used for Week 1's Object Types and properties applies directly to tools.

Fewer, well-scoped tools beat many overlapping ones

An agent with ten narrow, clearly-described tools plans more reliably than one with three overloaded, multi-purpose ones.

Full Agent Architecture

How planning, memory, and tool invocation fit together in a single run



Every arrow in this diagram is a point where governance applies: which tools the Planner may even see (RBAC/ABAC), what the Tool Selector is permitted to call, and what the Observe step is allowed to retain in memory.

Guardrails: Bounding What an Agent Can Do

Autonomy inside a fence, not autonomy without one

Tool-level access control

An agent only ever sees the Functions and Actions its role is granted — the same RBAC/ABAC model from Week 1, applied to tools instead of objects.

Approval gates on irreversible actions

The human-in-the-loop approval pattern from Week 3 doesn't disappear — it becomes the condition that stops an agent's plan and waits, for exactly the actions your governance plan marks as sensitive.

Bounded run length and scope

An agent given no limit on how many steps it can take, or how broad its goal is, is a runaway-cost and runaway-risk problem — the same discipline as bounding a pipeline's compute in Foundry.

Error Handling for Agents

A failed tool call is a normal event an agent must plan around, not an exception that halts everything

Distinguish a retryable failure from a hard stop

A transient search timeout can be retried; a Function returning “policy not found” is a signal to change the plan, not to retry the same call.

Compensating actions, not silent rollback

The same compensating-transaction pattern from Week 3's writeback discussion applies here — an agent that partially completes a multi-step plan needs a defined way to undo or flag what it already did.

Every failure is logged, not swallowed

An agent that fails quietly is worse than a workflow that fails loudly — Week 6's observability and audit-trail topics depend on every tool call, success or failure, being recorded.

TOPIC 3

Enterprise Agent Design Patterns

Event-driven triggers, microservices-style isolation, and Agent Studio

Event-Driven Agents: What Triggers a Run

An agent doesn't run continuously — something specific wakes it up

Object-change events

A new Claim object created, or a Risk Assessment's status changing to “Referred” — the same Ontology events your Automate discussions in Week 3 were building toward.

Scheduled events

A Policy Recommendation Agent that runs nightly against policies approaching renewal, rather than waiting for a person to ask.

Upstream agent events

One agent's output becoming another agent's trigger — the Fraud Detection Agent's flag becoming the event that wakes the Compliance Agent, previewing Day 2's collaboration model.

Microservices-Style Agent Isolation

Treating each agent like an independently deployable service, not a shared monolith

Each agent owns its scope, and only its scope

A Claims Agent doesn't need underwriting data any more than a claims microservice would need direct access to an underwriting database.

One agent's failure doesn't take down another

If the Fraud Detection Agent's model needs retraining, the Underwriting Agent keeps running — the same isolation benefit a microservices architecture gives application services.

Agents version and deploy independently

Updating the Compliance Agent's rules shouldn't require redeploying the Claims Agent — independent release cycles are a design goal, not an accident.

Enterprise Design Patterns Applied to Agents

The same patterns from earlier weeks, now applied to autonomous systems

Pattern	Where You've Seen It Before	How It Applies to Agents
Event-driven architecture	Week 3 — Automate triggers on Ontology events	An agent run is triggered by an event, not polled for on a schedule by default
Microservices-style isolation	General enterprise architecture practice	Each agent is scoped, deployed, and secured independently
Least-privilege access	Week 1 — RBAC / ABAC on objects and properties	An agent sees only the tools and data its role requires
CQRS (read vs. write separation)	Week 3 — Ontology reads vs. Action writebacks	An agent's reasoning (read) is separated from its committed writeback (Action submission)

Agent Studio: Where These Concepts Live in AIP

You are not building this from scratch — Agent Studio is the workbench for exactly this

Agent Studio configures the reasoning loop

The goal, the available tools, and the model behind an agent are configured here — conceptually the same Use LLM configuration from Week 2's AIP Logic blocks, extended with tool access.

Tools are the same Functions and Actions you already built

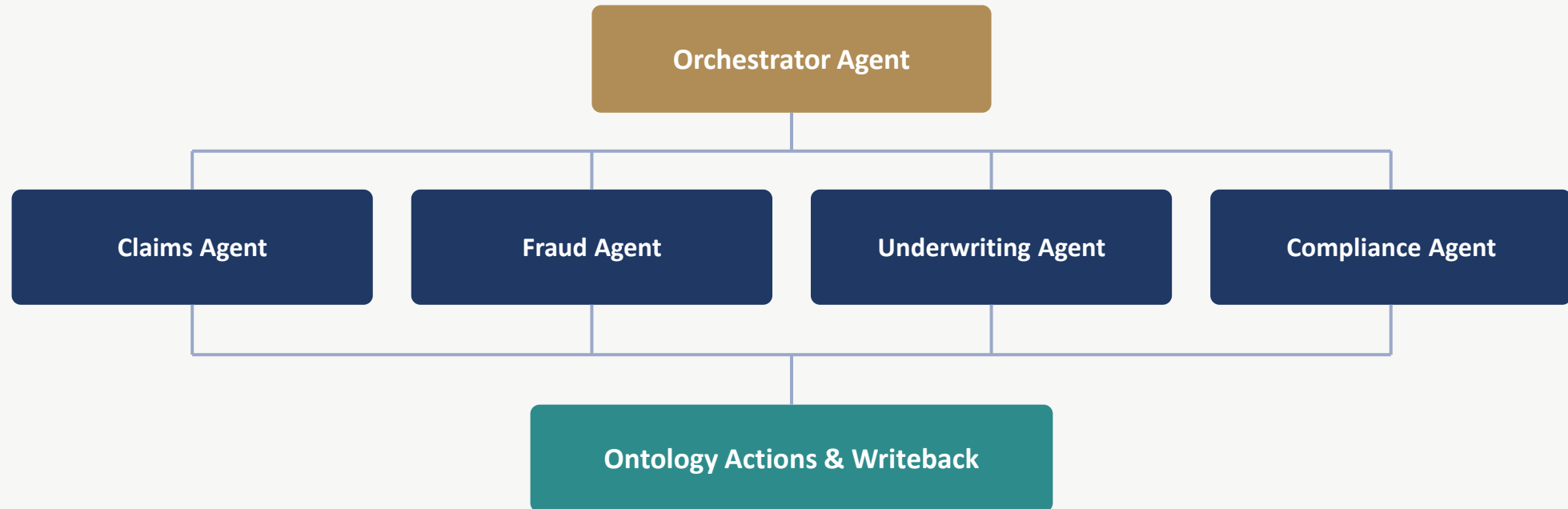
Nothing new needs to be created for an agent to use a Week 3 Function — it is exposed to the agent as an available tool, with its existing description and permissions.

Wednesday's lab configures, it doesn't code from zero

You will assign your batch's two agents in Agent Studio, wire them to existing Functions and Actions, and define their triggers — the mechanics, not the underlying platform, are new.

Orchestration Preview: The Pattern Wednesday Builds

One orchestrator, several specialized agents, one shared writeback path



Day 2 covers how the Orchestrator Agent decides which specialized agent to invoke and how results are combined — today's job is recognizing each box as something you already understand on its own.

TOPIC 4

Insurance Agent Roles

The specialized agents your batch's lab will orchestrate on Day 3

Why Specialize Instead of Building One Large Agent

Narrow, single-responsibility agents that collaborate beat one agent that does everything

Easier to evaluate

A Fraud Detection Agent's behavior can be scored on fraud-detection outcomes alone — Week 6's Evals become far harder to design against one agent handling five unrelated responsibilities.

Easier to secure

A narrow agent needs a narrow set of tools and data access — the same least-privilege principle behind Week 1's object-level security policies, applied to what an agent is allowed to see and call.

Easier to isolate and deploy independently

One agent's changes or failures don't require redeploying every other agent — the same reasoning behind a microservices architecture, applied to AI components instead of application services.

The Insurance Agents in This Program

Seven specialized roles — Wednesday's lab assigns two per batch, based on your use case

Agent	Responsibility
Claims Agent	Coordinates claim intake through settlement recommendation
Underwriting Agent	Evaluates new-business applications against rating factors
Fraud Detection Agent	Flags anomalous claims for SIU review
Risk Assessment Agent	Scores applicant and policy risk on an ongoing basis
Compliance Agent	Checks a proposed action against regulatory and policy constraints
Policy Recommendation Agent	Recommends coverage, renewal, or retention offers
Customer Support Agent	Handles policyholder-facing queries and routes escalations

Agent Deep Dive: Claims Agent & Fraud Detection Agent

Batch 1's focus — the two agents behind Wednesday's Claims Operations lab

Claims Agent — goal and tools

Goal: move a claim from intake to a settlement recommendation. Tools: Coverage Validation Function, Claim History search, Settlement Function — all built in earlier weeks.

Fraud Detection Agent — goal and tools

Goal: flag anomalous claims before settlement. Tools: pattern-matching Function over claim history, Semantic search across prior SIU findings, a Referral Action.

Where they collaborate

The Claims Agent's plan includes a step that hands off to the Fraud Detection Agent whenever claim amount or pattern crosses a threshold — previewed in today's walked-through scenario.

Agent Deep Dive: Underwriting Agent & Risk Assessment Agent

Batch 2's focus — the two agents behind Wednesday's Underwriting & Risk lab

Underwriting Agent — goal and tools

Goal: recommend an underwriting decision for a new application. Tools: Rating Factor lookup, Risk Assessment retrieval, an Underwriting Decision Action.

Risk Assessment Agent — goal and tools

Goal: keep an applicant's risk score current as new information arrives. Tools: the same isHighRiskApplication derivation pattern from Week 1, Day 4, now invoked by the agent instead of a fixed pipeline.

Where they collaborate

The Underwriting Agent requests a fresh score from the Risk Assessment Agent before finalizing a decision, rather than trusting a stale, previously-computed value.

Agent Deep Dive: Compliance Agent, Policy Recommendation & Customer Support Agents

Batch 3 and Batch 4's focus areas

Compliance Agent — goal and tools (Batch 3)

Goal: check a proposed Fraud Detection Agent action against regulatory and internal policy constraints before it reaches a human. Tools: policy-rule lookup, an approval-gate Action.

Policy Recommendation Agent — goal and tools (Batch 4)

Goal: recommend coverage, renewal terms, or retention offers ahead of a policy's expiry. Tools: renewal-offer lookup (the same retention_offers dataset from your program materials), a Recommendation Action.

Customer Support Agent — goal and tools (Batch 4)

Goal: resolve policyholder-facing queries directly, or route to a human when the query exceeds its authority. Tools: knowledge retrieval from Week 2's Copilot pattern, an escalation Action.

Your Batch's Agent Focus on Day 3

Each batch orchestrates the two agents most relevant to its use case

Batch	Use Case	Primary Agents
Batch 1 — India	Claims Operations	Claims Agent + Fraud Detection Agent
Batch 2 — India	Underwriting & Risk Assessment	Underwriting Agent + Risk Assessment Agent
Batch 3 — Europe	Fraud & SIU	Fraud Detection Agent + Compliance Agent
Batch 4 — Europe	Policy Servicing	Policy Recommendation Agent + Customer Support Agent

Common Pitfalls

Patterns worth avoiding before Wednesday's lab



Calling any Copilot an “agent”

A retrieval-and-answer loop with no planning or tool invocation is still a copilot — reserve “agent” for a system that decides its own next step.



Giving an agent long-term memory by default

Persisted memory across runs is a deliberate design choice with its own governance cost, not a free upgrade — default to short-term, per-run memory unless a use case needs otherwise.



Skipping the plan → observe step after a tool call

An agent that fires a Function and moves on without checking the result behaves like a fixed script with extra steps, not an agent.

Common Pitfalls

Patterns worth avoiding before Wednesday's lab (cont'd)



Building one large agent instead of specialized ones

A single agent trying to do claims, fraud, and underwriting together is harder to evaluate, harder to secure, and harder to debug than four narrow agents that collaborate.



Treating every event as worth waking an agent for

An event-driven agent that fires on every minor Ontology change wastes compute and floods logs — trigger conditions need the same scrutiny as a security policy condition.



Assuming an agent's plan is always correct because it ran without error

No error does not mean the right decision — Week 6's Evals exist precisely because an agent can complete a plan cleanly and still reach the wrong outcome.

Recap & What's Next

What We Covered Today

- The difference between a copilot, a fixed workflow, and an agent
- The plan → act → observe loop, walked through a real claim scenario
- Planning, short-term memory, long-term memory, and tool invocation
- Guardrails and error handling for agents that call Functions and Actions
- Event-driven triggers and microservices-style agent isolation
- Where these ideas live inside AIP's Agent Studio
- The seven specialized insurance agents, and your batch's Day 3 focus

Day 2 — Agent Collaboration & Multi-Agent Orchestration